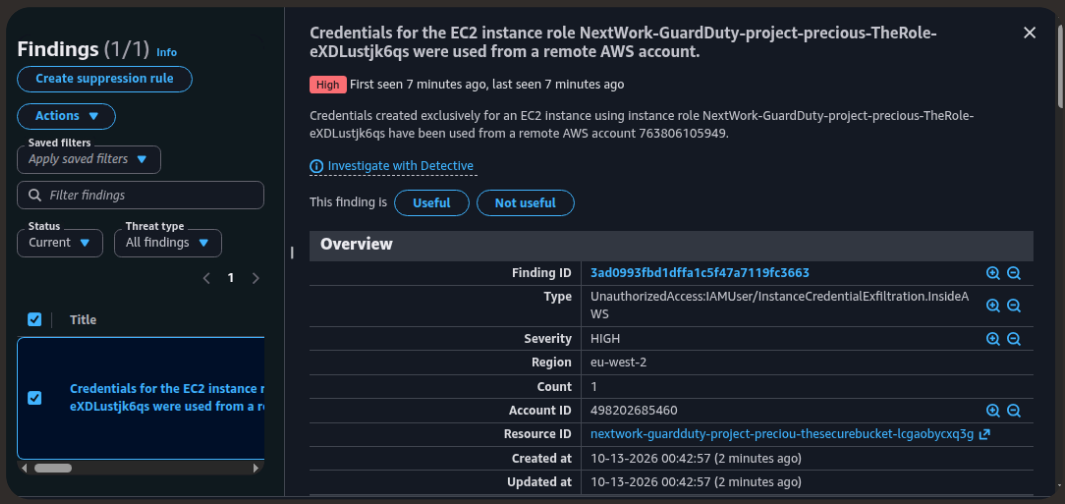
THREAT DETECTION WITH GUARDDUTY

By Precious Awoyemi | October 2026



Introducing Today's Project!

Tools and concepts

The services I used were AWS CloudFormation and Amazon GuardDuty. Key concepts I learned include SQL and command injection, change sets in CloudFormation, enabling and configuring GuardDuty for threat detection, understanding its finding categories, and integrating alerts with automated remediation for centralized security monitoring.

Project reflection

This project took approximately 2 hours, including demo time. The most challenging part was ensuring GuardDuty was correctly integrated and automated through CloudFormation without misconfigurations. The most rewarding part was seeing a fully automated security monitoring setup deployed successfully with minimal manual intervention.

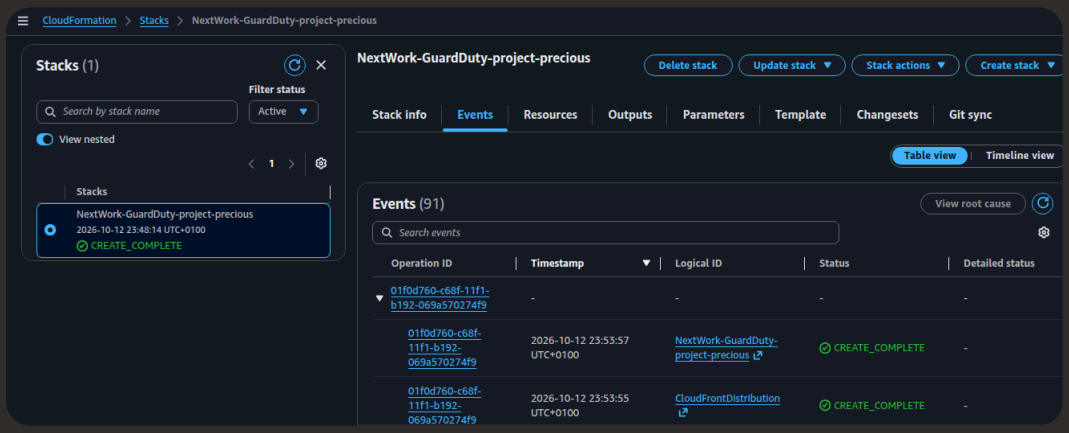
I completed this project today to strengthen my skills in automating AWS security services and integrating threat detection, as well as understanding its importance in cloud security and engineering. It met my goals by giving me hands-on experience with CloudFormation and GuardDuty while achieving a fully functional, automated security monitoring setup.

Project Setup

To set up this project, I deployed a CloudFormation template that launches an intentionally insecure web app (OWASP Juice Shop). The three main components are the web app infrastructure, an S3 bucket for storage, and GuardDuty for security monitoring.

The web app I deployed is OWASP Juice Shop, an intentionally insecure application built with many security flaws for learning and practice. To build my GuardDuty skills, I will simulate attacks against the app, then review the GuardDuty findings and analyze what it detected.

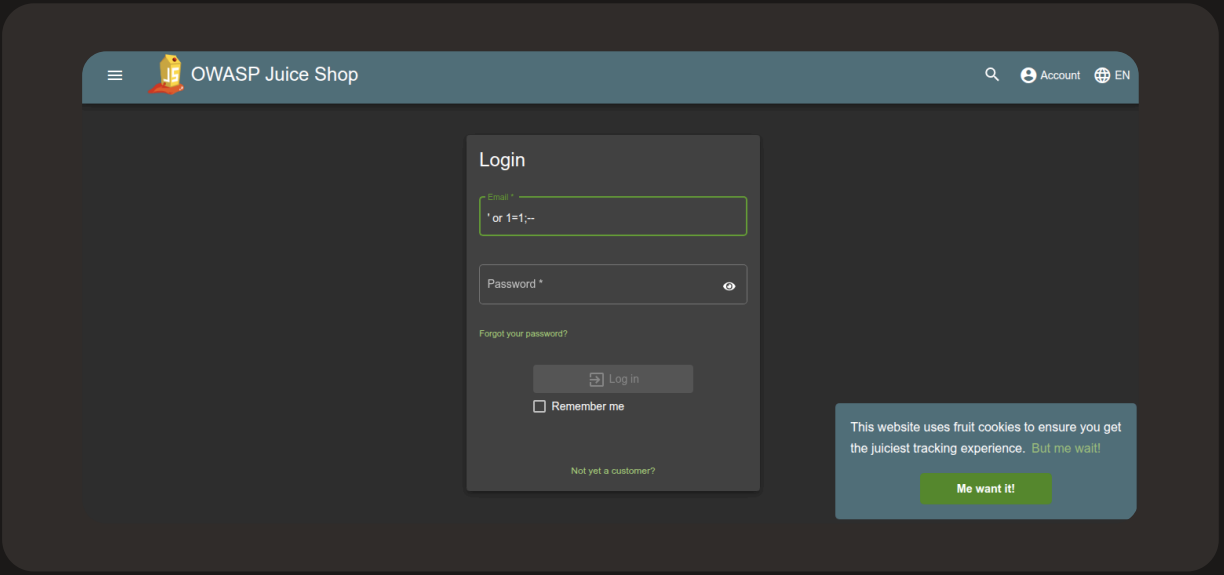
GuardDuty is an AWS threat detection service that uses machine learning, anomaly detection, and threat intelligence to continuously monitor your AWS account and resources for suspicious or malicious activity. In this project, it will help me detect potential security threats affecting the AWS resources that host my deployed web app, such as the EC2 instance.



SQL Injection

The first attack I performed on the web app was SQL injection, where malicious SQL code is entered into input fields to manipulate the database. SQL injection is a serious security risk because it can let attackers read, modify, or delete sensitive data without authorization

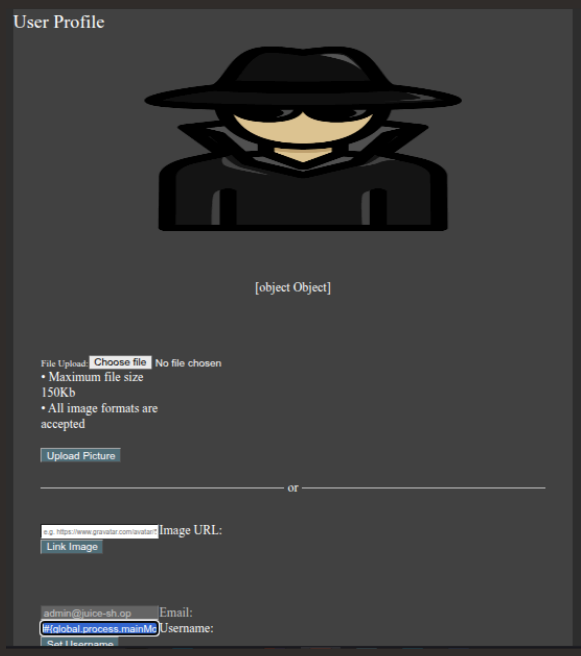
My SQL injection attack involved entering ' or 1=1;-- into the email field of the login page. The single quote closes the email string in the query, or 1=1 makes the condition always evaluate to true, and -- comments out the rest of the query, including the password check. This lets the attacker bypass authentication



Command Injection

Next, I used command injection, an attack that lets an attacker execute arbitrary system commands on the server through a vulnerable application. The Juice Shop web app is vulnerable to this because it does not properly sanitize or validate user input before passing it to system functions.

To run command injection, i entered the malicious payload into the username field of the admin panel in OWASP Juice Shop. The injected command executed on the server, and in this case, it successfully triggered the attack by altering the user profile name. The profile name changed to [object Object], indicating that the injected command executed and returned a non-string object, which confirms the command injection was successful.



Attack Verification

To verify the attack's success, I visited the publicly exposed credentials file at https://d208toc1ekxdpb.cloudfront.net/assets/public/credentials.json, which is where the AWS access keys were exposed. The page revealed the access keys that represent my EC2 instance's access to my AWS environment. Anyone who found this file could use those keys to gain the same level of access.

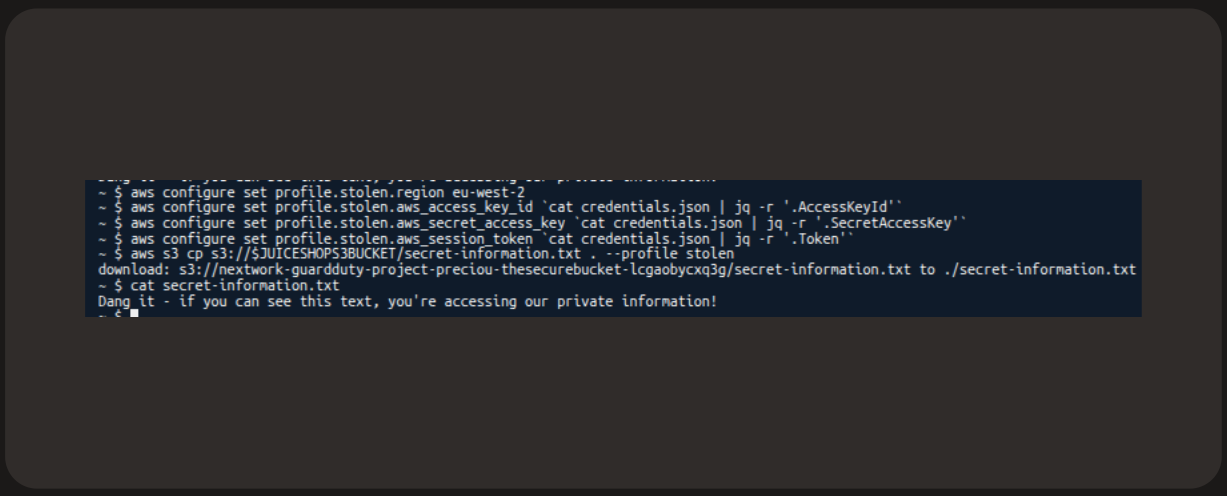

```
{
  "AccessKeyId": "ASIAXH7ZRWAKDSM5VAYA",
  "Code": "Success",
  "Expiration": "2026-10-13T05:26:18Z",
  "LastUpdated": "2026-10-12T22:50:22Z",
  "SecretAccessKey": "tdyppl3ZK3YoM67jdghqK6ToKeYti13NXI3yXt7a",
  "Token":
"IQoJb3JpZ2luX2VjEMf//////////wEaCWV1LXdlc3QtMiJHMEUCIFJMajWGnkwrMWJif+r4VD5xHg0UJTqGd1I
yMDI2ODU0NjAiDOxCbmFcrMSDDavTkCqWBf7xdkcqwNPxcMitvvKO0Gbnm0rklnYcUwPeCSmEqDsAvLmlQ89HhBI
nrR54yTvTfy9WwAUu+SlwAj62EnuPLCgSd9HCLOSgKE7y77zP+bNkjBKW665WiWXlsIuqAwRz2xug3oQma0Cj2mv
pEKOZyDCpL+aqReOkuFaMOA0JJNYlfTZ3b4ocEdaHzeGo1lbN8PWE6o2qtJCO38ts8BPuIxhKYOh/sk+tpT8KjPT
jKEjCfT7ysPOzzlk6M9yUlS1MlteQ2o4zpyBhCpOPNvJJBh/DfGeAhu7CVJjrm/ciCWJgtHDGtRWs8IX+PC5Kt8r
5LbNZyrssoDhfKOH6zHYr0L4mnxeB+Bpc7J80QOAHFvpve35QK7x0pAaVrEVKfUPVLHHP6nxX5lJhKe0tx8OXm9I
cG9PryuHCJL0bEYJtCf4EiuPlpKq5yMEizr4SZHXsnccL3PUEnPQTzykhMObItdYGOrEBAa3OiuelwmcSXC4ACk
DfDRuKdiBx2bThoOnf+mL7A/3x5H/dvY7ELN51fDRRTtdit8yYMN3fEKshMCdGDLzywG6qfVJcmIL0efwFlr9W3I
  "Type": "AWS-HMAC"
}
```

Using CloudShell for Advanced Attacks

The attack continues in CloudShell because it provides a browser-based command-line environment within AWS, where I can quickly configure the AWS CLI using the stolen credentials. This allows me to simulate how an attacker could use those credentials to access AWS resources directly from within the AWS environment, with no local setup required, such as copying sensitive files from an S3 bucket.

In CloudShell, I used wget to download the credentials.json file exposed by the vulnerable web app. I then used cat and jq to parse and format the JSON data into a human-readable form, making the exposed AWS credentials easy to view clearly.

I then set up a separate AWS CLI profile, named stolen, to configure the CLI with the compromised credentials. Creating a new profile let me use the stolen credentials without affecting my default AWS configuration, and it also simulated how an attacker would operate: using a distinct identity to access AWS resources rather than the account's normal credentials.

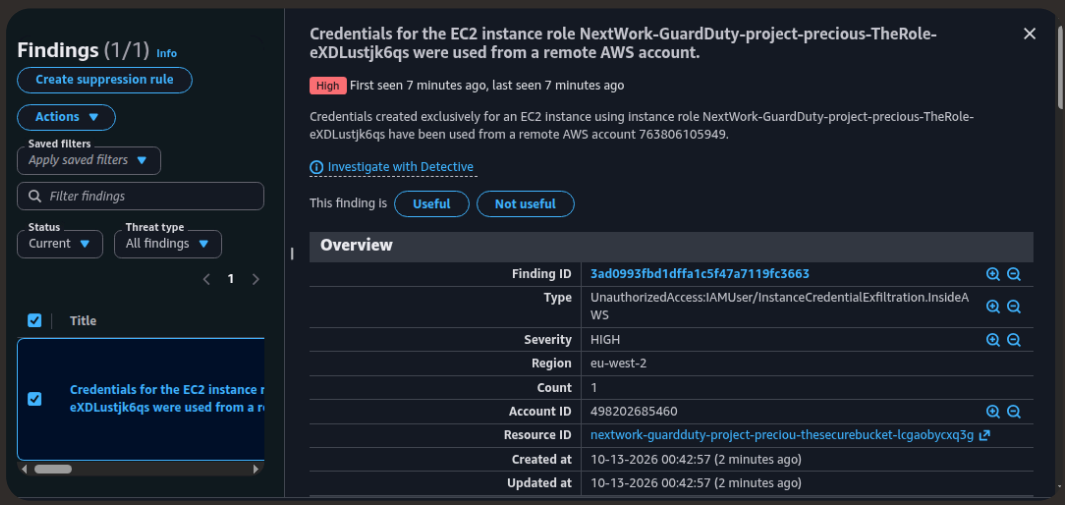


GuardDuty's Findings

After performing the attack, GuardDuty reported a finding within five minutes. Findings are notifications from GuardDuty indicating that something suspicious has happened, providing additional details on who, how, and when of the attack.

GuardDuty's finding was called UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.InsideAWS, which means there was unusual use of your EC2 instance's credentials from within AWS, suggesting they may have been compromised and used by an unauthorized entity. Anomaly detection was used because GuardDuty continuously learns what normal behavior looks like in your AWS environment and flagged this activity as abnormal based on unexpected access patterns.

GuardDuty's detailed finding reported that the S3 bucket was affected. The action taken using the stolen credentials was GetObject, and the finding identified the EC2 instance whose credentials were leaked. The IP address and location of the actor were also available.



Extra: Malware Protection

For my project extension, I enabled malware protection for S3 in GuardDuty. Malware is malicious software designed to harm, exploit, or compromise systems and data, so enabling this protection helps detect malware hidden in files uploaded to S3 buckets.

To test Malware Protection, I uploaded a test file to the S3 bucket (the EICAR test file, an industry-standard file used to verify antivirus detection). The file won't actually cause damage because it's a safe, harmless sample designed specifically to trigger detection without harming the system or data.

Once I uploaded the file, GuardDuty instantly triggered a malware detection finding. This confirmed that GuardDuty's malware protection effectively identifies and alerts on potentially harmful files stored in S3, helping to safeguard the environment from malicious threats.

